

CS20202: Software Engineering

Static Analysis

Soumyajit Dey, Jibesh Patra,
CSE Dept., IIT Kharagpur

**CS20202:
Software
Engineering**

Soumyajit Dey,
Jibesh Patra,
CSE Dept., IIT
Kharagpur

Introduction

Dataflow Analysis

Motivation

Program analysis aims to automatically determine various properties of programs. Applications include:

- Compiler optimisation **efficiency w.r.t. time and space**
- Bug detection **catch errors / potential issues early**
- Security verification
- Resource analysis

Key challenge:

Precise analysis is "undecidable" in many cases

Therefore we use something called ****safe approximations****. : More on this later
¹

¹In many places, slides have contents reproduced from "Principles of Program Analysis - Flemming Nielson; Hanne Riis Nielson; Chris Hankin"

Static vs Dynamic Analysis

An analyzer is yet another program, can be part of compiler infra

Dynamic Analysis

- Observes program during execution
- Requires test inputs

Static Analysis

- Analyzes program without executing
- Considers all possible inputs

Currently we shall focus on :

Basics of Static Analysis

Concrete Semantics

Let:

$$States = Var \rightarrow Values$$

A program statement defines a transition:

$$S : States \rightarrow States$$

Example:

$$x = x + 1$$

If state is

$$\sigma(x) = 4$$

then

$$S(\sigma)(x) = 5$$

Concrete semantics describes **exact execution behavior**.

Why Exact Analysis is Impossible

Many program properties are undecidable. Example:

Does program P terminate? Can some program $S(P)$ decide that.

Note: S runs P does not

This is equivalent to the famous *Halting Problem* of Turing Machines.

Therefore, in general:

Exact static analysis is impossible

Solution:

Approximate analysis

Abstract Interpretation Idea

Instead of computing exact states:

Concrete domain

we compute:

Abstract domain

Example: Concrete value:

$$x = 5$$

Abstract value:

$$x > 0$$

We sacrifice precision for **computability**.

Concrete Domain

Concrete domain contains exact program states.

Example:

$$\sigma : Var \rightarrow \mathbb{Z}$$

Example state:

$$\sigma(x) = 3, \quad \sigma(y) = -1$$

Concrete domain:

$$D_c = Var \rightarrow \mathbb{Z}$$

This domain is usually ****infinite****.

Abstract Domain

Abstract domain represents sets of concrete states. Example abstraction:

$$Sign = \{neg, zero, pos\}$$

Mapping example:

$$5 \rightarrow pos$$

$$-3 \rightarrow neg$$

Abstract domain:

$$D_a = Var \rightarrow Sign$$

Now analysis becomes finite.

Abstraction and Concretization

Two important functions:

Abstraction:

$$\alpha : D_c \rightarrow D_a$$

Concretization:

$$\gamma : D_a \rightarrow 2^{D_c}$$

Example:

$$\alpha(5) = pos$$

$$\gamma(pos) = \{1, 2, 3, \dots\}$$

These relate abstract and concrete semantics.

Partial Orders and Lattices

Abstract domains usually form ****lattices****. Example sign lattice:

$$\{neg, zero, pos\}$$

with top:

$$\top$$

and bottom:

$$\perp$$

Join operator:

$$a \sqcup b$$

Example:

$$pos \sqcup neg = \top$$

Program Example

Consider program:

```
x = 1
y = x + 1
if y > 0 then
  z = y
else
  z = -y
```

Concrete value:

$$z = 2$$

Abstract sign analysis:

$$z = \textit{pos}$$

We lose precision but keep safety.

Control Flow Graph

Program is represented as:

$$CFG = (N, E)$$

Where:

- N = nodes (program points)
- E = edges (control flow)

Example:

$$n_1 \rightarrow n_2 \rightarrow n_3$$

Dataflow information propagates along edges.

The While Programming Language

The **While language** is a simple imperative language used to illustrate program analysis techniques. Syntax of statements:

$$\begin{aligned} S ::= & x := a \\ & | \textit{skip} \\ & | S_1; S_2 \\ & | \textit{if } b \textit{ then } S_1 \textit{ else } S_2 \\ & | \textit{while } b \textit{ do } S \end{aligned}$$

where

- $x \in \textit{Var}$ (variables)
- $a \in \textit{AExp}$ (arithmetic expressions)
- $b \in \textit{BExp}$ (boolean expressions)

Dataflow Analysis Idea

At each node we compute information:

$$IN[n]$$

$$OUT[n]$$

Transfer function:

$$OUT[n] = f_n(IN[n])$$

Join at merges:

$$IN[n] = \bigsqcup_{p \in \text{pred}(n)} OUT[p]$$

This forms a ****system of equations****.

Example: Reaching Definitions

Definition reaches a program point if:

no redefinition occurs along the path

Example:

```
1: x = 1
2: y = 2
3: x = 3
4: print(x)
```

At line 4: Reaching definitions:

$\{3\}$

Definition at line 1 does not reach.

Example: Live Variable Analysis

Variable is live if its value is used later.

Example:

```
x = 1  
y = x + 2  
print(y)
```

Before line 2:

Live variables:

 $\{x\}$

Before line 3:

 $\{y\}$

Fixpoint Computation

Dataflow equations solved using iteration.

Initialize:

$$IN[n] = \perp$$

Iteratively apply transfer functions until:

$$IN_{i+1} = IN_i$$

This is called a **fixpoint**.

Guaranteed to terminate if domain is finite.

Applications

Program analysis is used in:

- Compilers
- Static analyzers
- Security tools
- Parallelization

Examples:

- Dead code elimination
- Constant propagation
- Alias analysis

Summary

Key ideas:

- Concrete semantics describes exact execution
- Abstract domains approximate behavior
- Lattices structure abstract information
- Dataflow analysis propagates information over CFG
- Fixpoint algorithms compute analysis results

Motivation

Program analysis aims to determine properties of programs without executing them.

Motivation

Program analysis aims to determine properties of programs without executing them.

Examples:

- Detect uninitialized variables
- Dead code detection
- Compiler optimization
- Security verification

Motivation

Program analysis aims to determine properties of programs without executing them.

Examples:

- Detect uninitialized variables
- Dead code detection
- Compiler optimization
- Security verification

Dataflow Analysis studies how information flows through the program.

Basic Idea of Dataflow Analysis

Programs consist of statements that transform program state.

Basic Idea of Dataflow Analysis

Programs consist of statements that transform program state.

We want to compute information such as:

- Which variables are live?
- Which definitions reach a program point?
- Which expressions are already computed?

Basic Idea of Dataflow Analysis

Programs consist of statements that transform program state.

We want to compute information such as:

- Which variables are live?
- Which definitions reach a program point?
- Which expressions are already computed?

This is done by propagating information along the **control flow graph**.

Control Flow Graph (CFG)

A Control Flow Graph represents program structure.

CS20202:
Software
Engineering

Soumyajit Dey,
Jibesh Patra,
CSE Dept., IIT
Kharagpur

Introduction

Dataflow Analysis

Control Flow Graph (CFG)

A Control Flow Graph represents program structure.

Nodes:

- program statements or basic blocks

Control Flow Graph (CFG)

A Control Flow Graph represents program structure.

Nodes:

- program statements or basic blocks

Edges:

- possible flow of control

Control Flow Graph (CFG)

A Control Flow Graph represents program structure.

Nodes:

- program statements or basic blocks

Edges:

- possible flow of control

Example:

if($x > 0$) $y = 1$; *else* $y = 2$;

creates a branch in the CFG.

Forward vs Backward Analysis

Two main kinds of dataflow analysis.

CS20202:
Software
Engineering

Soumyajit Dey,
Jibesh Patra,
CSE Dept., IIT
Kharagpur

Introduction

Dataflow Analysis

Forward vs Backward Analysis

Two main kinds of dataflow analysis.

Forward Analysis

Information flows in the same direction as program execution.

Examples:

- Reaching Definitions
- Constant Propagation

Forward vs Backward Analysis

Two main kinds of dataflow analysis.

Forward Analysis

Information flows in the same direction as program execution.

Examples:

- Reaching Definitions
- Constant Propagation

Backward Analysis

Information flows opposite to execution.

Examples:

- Live Variable Analysis

Lattices

Dataflow information is modeled using a **lattice**.

Lattices

Dataflow information is modeled using a **lattice**.

A lattice consists of:

- Set of elements
- Partial ordering
- Least upper bound (join)
- Greatest lower bound (meet)

Lattices

Dataflow information is modeled using a **lattice**.

A lattice consists of:

- Set of elements
- Partial ordering
- Least upper bound (join)
- Greatest lower bound (meet)

Example lattice for sets:

$$(\mathcal{P}(D), \subseteq)$$

where D is a set of definitions.

Reaching Definition Analysis

Sample program written in the WHILE language computes the factorial of the number stored in x and leaves the result in z:

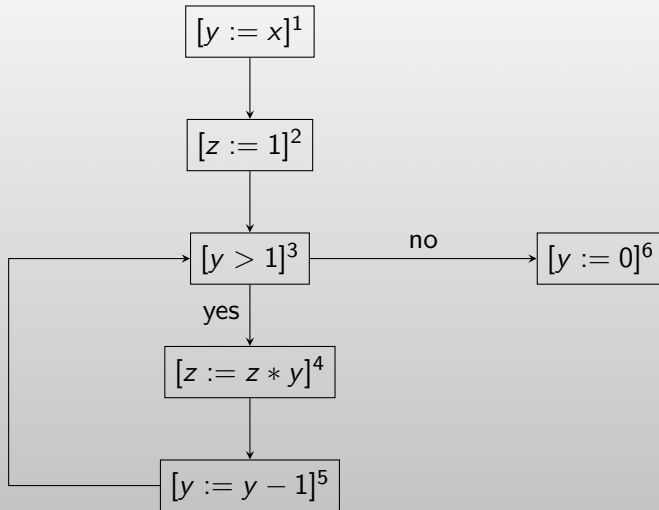
$[y := x]^1; [z := 1]^2; \textbf{while } [y > 1]^3 \textbf{ do } ([z := z * y]^4; [y := y - 1]^5); [y := 0]^6$

Reaching Definition/Assignment: An assignment (called a definition in the classical literature) of the form

$$[x := a]^l$$

may reach a certain program point (typically the entry or exit of an elementary block) if there is an execution of the program where x was last assigned a value at line 'l' when the program point is reached.

Reaching Definition Analysis



Reaching Definition Analysis

Full information about reaching definitions for the factorial program is given by the pair $RD = (RD_{entry}, RD_{exit})$

ℓ	$RD_{entry}(\ell)$	$RD_{exit}(\ell)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (y, 5), (z, 2), (z, 4)$	$(x, ?), (y, 1), (y, 5), (z, 2), (z, 4)$
4	$(x, ?), (y, 1), (y, 5), (z, 2), (z, 4)$	$(x, ?), (y, 1), (y, 5), (z, 4)$
5	$(x, ?), (y, 1), (y, 5), (z, 4)$	$(x, ?), (y, 5), (z, 4)$
6	$(x, ?), (y, 1), (y, 5), (z, 2), (z, 4)$	$(x, ?), (y, 6), (z, 2), (z, 4)$

For $l = 1$, y has been last defined in line 1, hence $(y, 1) \in RD_{exit}(1)$. For $l = 3$?

Building an equational system

$$RD_{exit}(1) = (RD_{entry}(1) \setminus \{(y, \ell) \mid \ell \in Lab\}) \cup \{(y, 1)\}$$

$$RD_{exit}(2) = (RD_{entry}(2) \setminus \{(z, \ell) \mid \ell \in Lab\}) \cup \{(z, 2)\}$$

$$RD_{exit}(3) = RD_{entry}(3)$$

$$RD_{exit}(4) = (RD_{entry}(4) \setminus \{(z, \ell) \mid \ell \in Lab\}) \cup \{(z, 4)\}$$

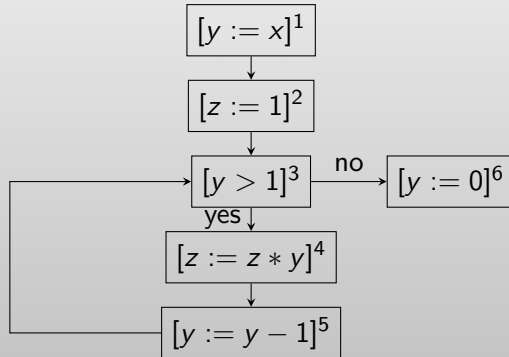
$$RD_{exit}(5) = (RD_{entry}(5) \setminus \{(y, \ell) \mid \ell \in Lab\}) \cup \{(y, 5)\}$$

$$RD_{exit}(6) = (RD_{entry}(6) \setminus \{(y, \ell) \mid \ell \in Lab\}) \cup \{(y, 6)\}$$

If a variable x is redefined in a line l , then previous definition is deleted and new definition (x, l) is added.

RD Analysis

Information collected from node exits need to connect with information entering successor node.



$$RD_{\text{entry}}(1) = \{(x, ?), (y, ?), (z, ?)\}$$

$$RD_{\text{entry}}(2) = RD_{\text{exit}}(1)$$

$$RD_{\text{entry}}(3) = RD_{\text{exit}}(2) \cup RD_{\text{exit}}(5)$$

$$RD_{\text{entry}}(4) = RD_{\text{exit}}(3)$$

$$RD_{\text{entry}}(5) = RD_{\text{exit}}(4)$$

$$RD_{\text{entry}}(6) = RD_{\text{exit}}(3)$$

So we have a set of equations with 12 variables, represented by $\overrightarrow{RD}$ such that overall transformation on the vector can be thought of as a functional map F and the solution of the equation should satisfy

$$\overrightarrow{RD} = F(\overrightarrow{RD})$$

See that any component of $\overrightarrow{RD}$ can be a collection of (variable, label)-s. Hence,

$$F : (\mathcal{P}(\text{Var}_* \times \text{Lab}_*))^{12} \rightarrow (\mathcal{P}(\text{Var}_* \times \text{Lab}_*))^{12}$$

where Var_* and Lab_* are subsets of Var and Lab containing program variables and line labels (including ? which means not yet defined) respectively.

RD Analysis

Let $(\mathcal{P}(\text{Var}_* \times \text{Lab}_*))^{12}$ be a set partially ordered by $\sqsubseteq$ defined as,

$$\overrightarrow{\text{RD}} \sqsubseteq \overrightarrow{\text{RD}'} \iff \forall i, \text{RD}_i \subseteq \text{RD}'_i$$

Complete lattice. A complete lattice $L = (L, \sqsubseteq) = (L, \sqsubseteq, \bigcup, \bigcap, \perp, \top)$ is a partially ordered set such that for all subsets $Y \subseteq L$ the least upper bound (lub) $\bigcup Y$ and the greatest lower bound $\bigcap Y$ exist. Furthermore,

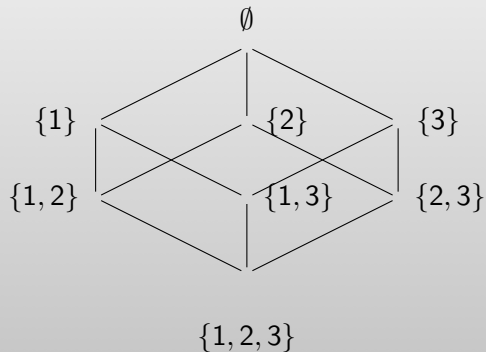
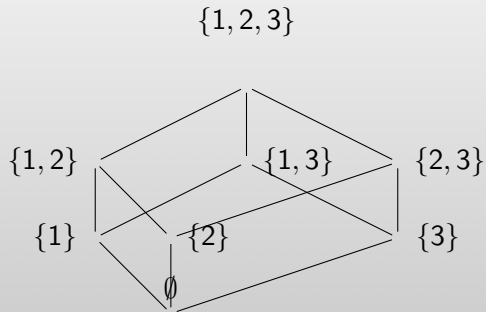
$$\perp = \bigcup \emptyset = \bigcap L$$

is the least element and

$$\top = \bigcup L = \bigcap \emptyset$$

is the greatest element.

RD Analysis



RD Analysis

Recall the set $(\mathcal{P}(\text{Var}_* \times \text{Lab}_*))^{12}$ partially ordered by $\sqsubseteq$,

$$\overrightarrow{RD} \sqsubseteq \overrightarrow{RD'} \iff \forall i, RD_i \subseteq RD'_i$$

where $\overrightarrow{RD} = (RD_1, \dots, RD_{12})$ and similarly $\overrightarrow{RD'} = (RD'_1, \dots, RD'_{12})$. It turns out that $(\mathcal{P}(\text{Var} \times \text{Lab}))^{12}$ is a complete lattice with least element $\bar{\theta} = (\emptyset, \dots, \emptyset)$ and least upper bounds given by

$$\overrightarrow{RD} \sqcup \overrightarrow{RD'} = (RD_1 \cup RD'_1, \dots, RD_{12} \cup RD'_{12})$$

It is easy to show that F is in fact a monotone function meaning that

$$\overrightarrow{RD} \sqsubseteq \overrightarrow{RD'} \text{ implies } F(\overrightarrow{RD}) \sqsubseteq F(\overrightarrow{RD'})$$

RD Analysis

This involves calculations like

$$RD_{\text{exit}}(2) \subseteq RD'_{\text{exit}}(2) \quad \text{and} \quad RD_{\text{exit}}(5) \subseteq RD'_{\text{exit}}(5)$$

It can be shown that

$$\Rightarrow RD_{\text{exit}}(2) \cup RD_{\text{exit}}(5) \subseteq RD'_{\text{exit}}(2) \cup RD'_{\text{exit}}(5)$$

$$\exists n \text{ such that, } \vec{\emptyset} \subseteq F(\vec{\emptyset}) \subseteq F^2(\vec{\emptyset}) \dots F^n(\vec{\emptyset}) = F^{n+1}(\vec{\emptyset})$$

Why? But if this is so then $F^n(\vec{\emptyset})$ is the solution of the EQ system.
The series terminates as the possible set of values is finite only.

$$F(\vec{\emptyset}) =$$

ℓ	$RD_{entry}(\ell)$	$RD_{exit}(\ell)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (y, 5), (z, 2), (z, 4)$	$(x, ?), (y, 1), (y, 5), (z, 2), (z, 4)$
4	$(x, ?), (y, 1), (y, 5), (z, 2), (z, 4)$	$(x, ?), (y, 1), (y, 5), (z, 4)$
5	$(x, ?), (y, 1), (y, 5), (z, 4)$	$(x, ?), (y, 5), (z, 4)$
6	$(x, ?), (y, 1), (y, 5), (z, 2), (z, 4)$	$(x, ?), (y, 6), (z, 2), (z, 4)$

- Since $\vec{\emptyset} = \emptyset^{12}$ is empty and application of F once gives a non empty $\vec{RD}$ component-wise; so $\vec{\emptyset} \sqsubseteq F(\vec{\emptyset})$ is true.
- by monotonicity of F , we have $F(\vec{\emptyset}) \sqsubseteq F^2(\vec{\emptyset})$ and so on.

- We are actually interested in the least solution of F
- If $\overrightarrow{RD}$ is another solution then $\overrightarrow{RD} = F(\overrightarrow{RD})$. However, note that,

$$\overrightarrow{\emptyset} \sqsubseteq \overrightarrow{RD}$$

$$\Rightarrow F(\overrightarrow{\emptyset}) \sqsubseteq F(\overrightarrow{RD})$$

$$\Rightarrow F^n(\overrightarrow{\emptyset}) \sqsubseteq F^n(\overrightarrow{RD}) = F^{n-1} \cdot F(\overrightarrow{RD}) = F^{n-1}(\overrightarrow{RD}) = \dots = \overrightarrow{RD}$$

- Hence $F^n(\overrightarrow{RD})$ is the least solution / the l.f.p. of F .

The Framework of Dataflow

Initial and final labels. When presenting examples of Data Flow Analyses we will use a number of operations on programs and labels. The first of these is

$$init : Stmt \rightarrow Lab$$

which returns the initial label of a statement:

$$init([x := a]^\ell) = \ell$$

$$init([\mathbf{skip}]^\ell) = \ell$$

$$init(S_1; S_2) = init(S_1)$$

$$init(\mathbf{if} \ b^\ell \ \mathbf{then} \ S_1 \ \mathbf{else} \ S_2) = \ell$$

$$init(\mathbf{while} \ b^\ell \ \mathbf{do} \ S) = \ell$$

We will also need a function

$$final : Stmt \rightarrow \mathcal{P}(Lab)$$

which returns the set of final labels in a statement. Whereas a sequence of statements has a single entry, it may have multiple exits (as for example in the conditional):

$$final(x := a^\ell) = \{\ell\}$$

$$final(\mathbf{skip}^\ell) = \{\ell\}$$

$$final(S_1; S_2) = final(S_2)$$

$$final(\mathbf{if} [b]^\ell \mathbf{then} S_1 \mathbf{else} S_2) = final(S_1) \cup final(S_2)$$

$$final(\mathbf{while} [b]^\ell \mathbf{do} S) = \{\ell\}$$

Note that the while-loop terminates immediately after the test has evaluated to false

Blocks

To access the statements or tests associated with a label in a program we use the function

$$blocks : Stmt \rightarrow \mathcal{P}(Blocks)$$

where *Blocks* is the set of statements, or elementary blocks, of the form $[x := a]^\ell$ or $[\mathbf{skip}]^\ell$ as well as the set of tests of the form $[b]^\ell$. It is defined as follows:

$$blocks([x := a]^\ell) = \{[x := a]^\ell\}$$

$$blocks([\mathbf{skip}]^\ell) = \{[\mathbf{skip}]^\ell\}$$

$$blocks(S_1; S_2) = blocks(S_1) \cup blocks(S_2)$$

$$blocks(\mathbf{if} [b]^\ell \mathbf{then} S_1 \mathbf{else} S_2) = \{[b]^\ell\} \cup blocks(S_1) \cup blocks(S_2)$$

$$blocks(\mathbf{while} [b]^\ell \mathbf{do} S) = \{[b]^\ell\} \cup blocks(S)$$

The set of labels occurring in a program is given by

$$labels : Stmt \rightarrow \mathcal{P}(Lab)$$

where

$$labels(S) = \{\ell \mid [B]^\ell \in blocks(S)\}$$

Clearly $init(S) \in labels(S)$ and $final(S) \subseteq labels(S)$.

Flows and reverse flows. Flow / Reverse-flow is the forward / back edge in a CFG.

We define a function

$$flow : Stmt \rightarrow \mathcal{P}(Lab \times Lab)$$

which maps statements to sets of flows given next

$$\text{flow}([x := a]^\ell) = \emptyset$$

$$\text{flow}([\text{skip}]^\ell) = \emptyset$$

$$\text{flow}(S_1; S_2) = \text{flow}(S_1) \cup \text{flow}(S_2) \cup \{(\ell, \text{init}(S_2)) \mid \ell \in \text{final}(S_1)\}$$

$$\begin{aligned} \text{flow}(\text{if } [b]^\ell \text{ then } S_1 \text{ else } S_2) &= \text{flow}(S_1) \cup \text{flow}(S_2) \\ &\cup \{(\ell, \text{init}(S_1)), (\ell, \text{init}(S_2))\} \end{aligned}$$

$$\begin{aligned} \text{flow}(\text{while } [b]^\ell \text{ do } S) &= \text{flow}(S) \\ &\cup \{(\ell, \text{init}(S))\} \\ &\cup \{(\ell', \ell) \mid \ell' \in \text{final}(S)\} \end{aligned}$$

Thus $\text{labels}(S)$ and $\text{flow}(S)$ will be a representation of the flow graph of S .

flow is used in the formulation of *forward analyses*. $init(S)$ is the (unique) entry node for the flow graph with nodes $labels(S)$ and edges $flow(S)$. Also

$$labels(S) = \{init(S)\} \cup \{\ell \mid (\ell, \ell') \in flow(S)\} \cup \{\ell' \mid (\ell, \ell') \in flow(S)\}$$

and for composite statements (meaning those not simply of the form $[B]^\ell$) the equation remains true when removing the $\{init(S)\}$ component.

- In order to formulate *backward analyses* we require a function that computes reverse flows.

$$flow^R : Stmt \rightarrow \mathcal{P}(Lab \times Lab)$$

It is defined by:

$$flow^R(S) = \{(\ell', \ell) \mid (\ell, \ell') \in flow(S)\}$$

In case $final(S)$ contains just one element that will be the unique entry node for the flow graph with nodes $labels(S)$ and edges $flow^R(S)$. Also

$$labels(S) = final(S) \cup \{\ell \mid (\ell, \ell') \in flow^R(S)\} \cup \{\ell' \mid (\ell, \ell') \in flow^R(S)\}$$

and for composite statements the equation remains true when removing the $final(S)$ component.

Simple assumptions: from entry node there are only fwd edges (no incoming).
from exit there are no outgoing edges : isolated entry/exit CFG.

Thank You